

ch13 & 14

1. a. 偏移值为0. 长度值为0

b. 将记录分成三部分. 第一部分是空位图.

第二部分是顺序存储非空属性的偏移和长度.

第三部分是非空属性值.

若非空, 依据三部分找出具体值.

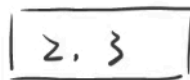
若为空, 则空位图中该属性为空. 返回空值.

2. (2, 3, 5, 7, 11, 17, 19, 23, 29, 31)

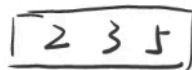
a. 1. 插入 2:



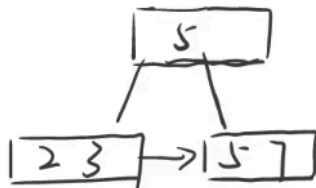
2. 插入 3



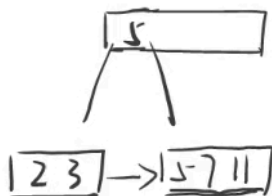
3. 插入 5



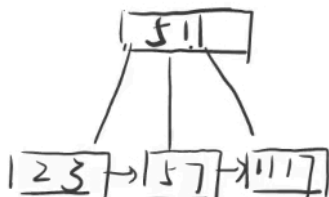
4. 插入 7:



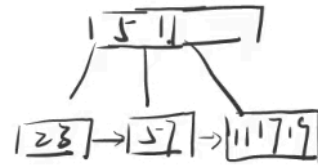
5. 插入 11



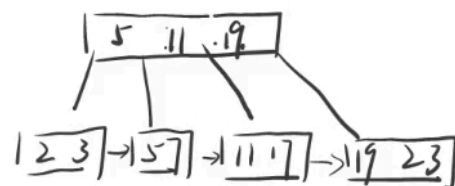
6. 插入 17



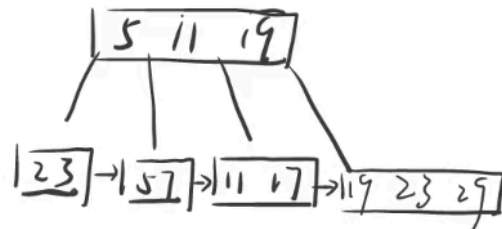
7. 插入 19



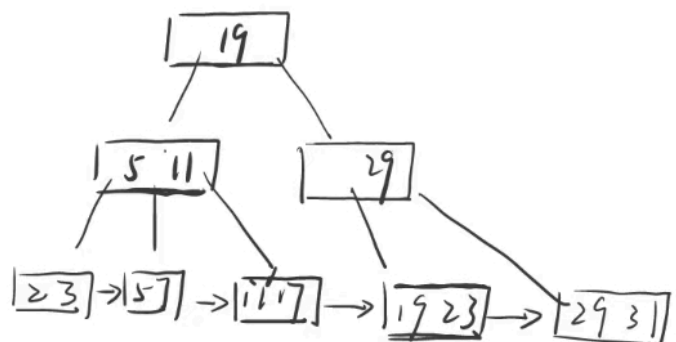
8. 插入 23



9. 插入 29



10. 插入 31



(2 3 5 7 11 17 19 23 29 31)

b. 1. 插入 2

12

2. 插入 3

123

3. 插入 5

1235

4. 插入 7

12357

5. 插入 11

1235711

6. 插入 17

17
1235 → 71117

7. 插入 19

17
1235 → 7111719

8. 插入 23

17
1235 → 711171923

9. 插入 29

1719
1235 → 71117 → 192329

10. 插入 31

1719
1235 → 71117 → 19232931

c. (2, 3, 5, 7, 11, 17, 19, 23, 29, 31)

1. 插入 2

12

2. 插入 3

12 3

3. 插入 5

12 3 5

4. 插入 7

12 3 5 7

5. 插入 11

12 3 5 7 11

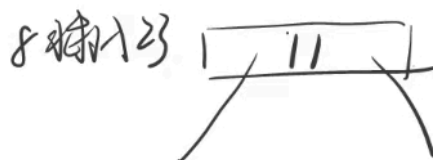
6. 插入 17

12 3 5 7 11 17

7. 插入 19

12 3 5 7 11 17 19

8. 插入 23



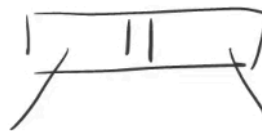
12 3 5 7 → 11 17 19 23

9. 插入 29



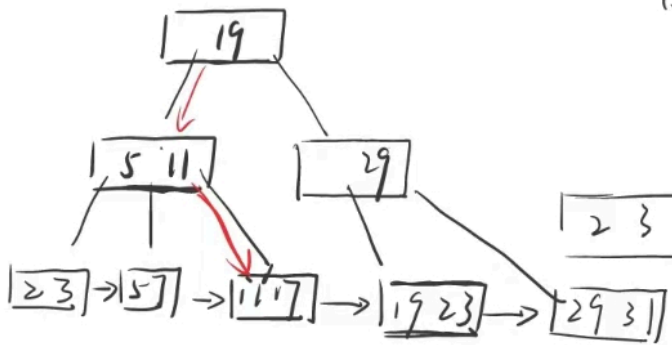
2 3 5 7 → 11 17 19 23 29

10. 插入 31



2 3 5 7 → 11 17 19 23 29 31

3. a. 指针为四个:

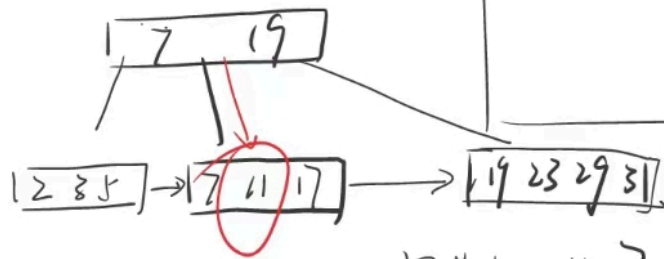


1. 第一次 I/O 访问根节点: 19
 19 > 11, 访问左边孩子节点

2. 第二次 I/O 访问左边孩子节点 (5, 11)
 比较后获取其最右边孩子节点

3. 第三次 I/O 访问叶子节点 (11, 17)
 找到 11, 访问结束

(1) 指针为六个



1. 第一次 I/O 访问根节点: 7 < 11 < 19
 获取中间孩子节点

2. 第二次 I/O 访问叶子 (7, 11) 遍历找到 11
 访问结束

指针为八个

(3)



[2, 3, 5, 7]

[11, 17, 19, 23, 29, 31]

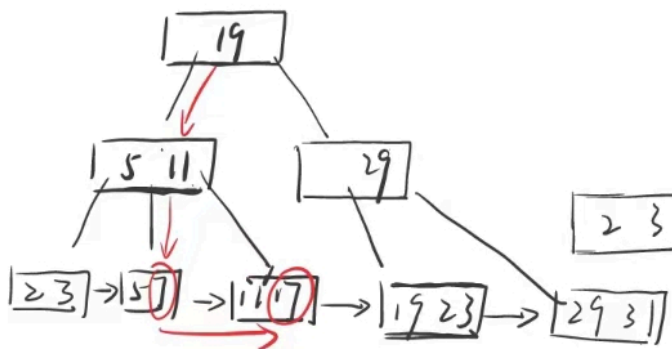
(1) 第一次 I/O 访问根节点
 获取最右边孩子节点

(2) 第二次获取叶子节点

(11, 17, 19, 23, 29, 31)

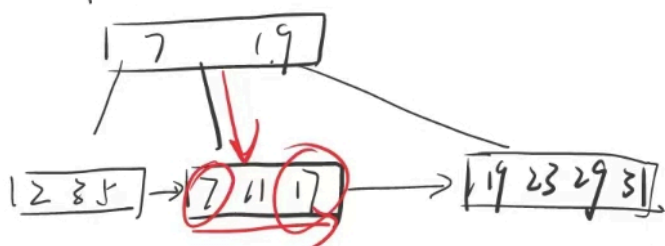
遍历得到 11
 访问结束

b. 指针数为4个



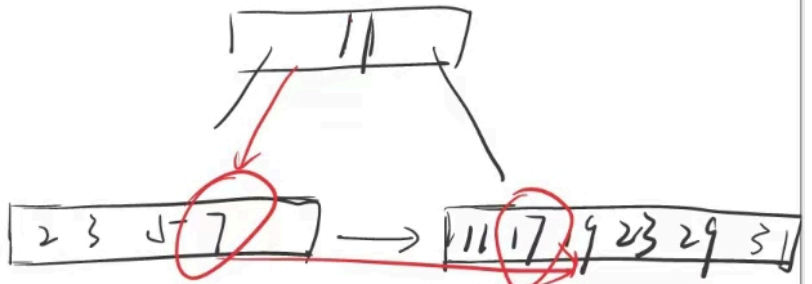
1. 第1次 I/O, 查找范围最左边点 7 位置, $19 > 7$, 访问左子树
2. 第2次 I/O, 访问 (5, 11), 找到中间叶子节点
3. 访问叶子节点 (5, 7) 找到 7, 然后遍历链表, 找出 [7, 17]

(2) 指针数为6个



1. 访问根节点, .. 返回中间子树
2. 访问叶子节点 (7, 11, 17), 找到 7, 遍历链表, 找出 [7, 17]

(3) 指针数为8个



- (1) 访问根节点
返回左子树
- (2) 访问叶子节点 (2, 3, 5, 7)
找到 7, 遍历链表,
找到 [7, 17]

4. 除最右边叶子节点之外, 其余叶子节点全部为半满状态。(升序排序)

因为插入操作总会在最右边叶子节点进行
所以当最右边叶子节点满后, 分裂成的非右子节点
不会有新值插入, 总是会保持半满状态